

MENDSPEECH REVISED LEARNING SYSTEM

Week 1: Audio, Degradation, and Measurement Foundations

Build the audio laboratory and make SpeechDamageBench a deterministic standalone package.

Six focused build days plus one consolidation day. Each day ends with named artifacts or measured evidence.

Week milestone

Build the audio laboratory and make SpeechDamageBench a deterministic standalone package.

Week map

Day	Focus	Minimum evidence	Compute
Day 1	Waveforms, sampling, and the MendSpeech baseline	• Compare 8 kHz, 16 kHz, 24 kHz, and 48 kHz versions by listening and plotting. • Measure duration, RMS energy, and file size for each version.	Local CPU
Day 2	Fourier intuition and STFT	• Hold audio constant and change one STFT setting at a time. • Write what phonetic or transient detail becomes easier or harder to see.	Local CPU
Day 3	Mel scale and log Mel features	• Change Mel bin count and compare visual structure and compute size. • Verify consistent feature shapes for different utterance lengths.	Local CPU
Day 4	Build SpeechDamageBench as a standalone package	• Generate mild, medium, and severe examples from the same clean sentence. • Reproduce the exact same damaged waveform from the same seed. • Change only the seed and verify that the corruption changes while all configured parameters remain fixed.	Local CPU
Day 5	Objective audio measurements	• Run all corruption levels on at least ten clips. • Look for cases where a metric disagrees with your listening judgment.	Local CPU
Day 6	Build the first MendSpeech audio console	• Test with three speakers and several corruption types. • Write down usability problems that would block later live debugging.	Local CPU
Day 7	Review, explain, and freeze Week 1	• From a blank notebook, recreate one corruption and one log Mel plot without copying previous cells.	Local CPU

Reference spine

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Waveforms, sampling, and the MendSpeech baseline

LEARN

- Waveform amplitude and time axes.
- Sampling rate, Nyquist intuition, bit depth, mono versus stereo.
- Why speech systems often standardize to 16 kHz.
- Duration, peak amplitude, RMS energy, and clipping.

BUILD IN MENDSPEECH

- Create the repository and a minimal audio loader.
- Record or collect five clean speech clips with consent.
- Normalize all clips to a consistent sample rate and mono format.

EXPERIMENT AND MEASURE

- Compare 8 kHz, 16 kHz, 24 kHz, and 48 kHz versions by listening and plotting.
- Measure duration, RMS energy, and file size for each version.

REQUIRED OUTPUT

- notebooks/day01_waveform.ipynb
- data/clean_manifest.csv
- docs/audio_baseline_notes.md

Completion check: You can explain what is lost when sample rate is reduced and can reproduce the same preprocessing from code.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Fourier intuition and STFT

LEARN

- Frequency, phase, harmonics, and spectral energy.
- Fourier transform intuition without memorizing derivations.
- STFT frames, window length, hop length, overlap.
- Tradeoff between time resolution and frequency resolution.

BUILD IN MENDESPEECH

- Implement STFT visualization with PyTorch or TorchAudio.
- Plot the same utterance with several window and hop settings.

EXPERIMENT AND MEASURE

- Hold audio constant and change one STFT setting at a time.
- Write what phonetic or transient detail becomes easier or harder to see.

REQUIRED OUTPUT

- notebooks/day02_stft.ipynb
- results/stft_parameter_grid.png

Completion check: You can choose a reasonable frame and hop configuration and explain why.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Mel scale and log Mel features

LEARN

- Human frequency perception and the Mel scale.
- Mel filterbanks and log compression.
- Number of Mel bins and dynamic range.
- Normalization of acoustic features.

BUILD IN MENDESPRECH

- Implement or inspect a log Mel feature pipeline.
- Build a function that returns features plus metadata needed for reproducibility.

EXPERIMENT AND MEASURE

- Change Mel bin count and compare visual structure and compute size.
- Verify consistent feature shapes for different utterance lengths.

REQUIRED OUTPUT

- `src/audio/features.py`
- `notebooks/day03_logmel.ipynb`
- `tests/test_features.py`

Completion check: You can trace waveform to STFT to Mel filterbank to log Mel tensor.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Build SpeechDamageBench as a standalone package

LEARN

- Deterministic corruption design and seed control.
- Additive noise, clipping, bandwidth limitation, dropouts, and reverberation.
- Why a benchmark should be reusable outside the main application.
- Versioned severity presets and manifest metadata.

BUILD IN MENDSPEECH

- Create an installable speechdamagebench package instead of burying corruptions inside MendSpeech.
- Implement noise, clipping, bandwidth reduction, dropout, and simple reverberation modules.
- Add a seed controlled configuration object and a small command line entry point.
- Record corruption name, severity, seed, parameters, and clean source id for every output.

EXPERIMENT AND MEASURE

- Generate mild, medium, and severe examples from the same clean sentence.
- Reproduce the exact same damaged waveform from the same seed.
- Change only the seed and verify that the corruption changes while all configured parameters remain fixed.

REQUIRED OUTPUT

- speechdamagebench/audio_damage.py
- speechdamagebench/presets.py
- speechdamagebench/cli.py
- pyproject.toml
- tests/test_determinism.py
- configs/damage_levels.yaml

Completion check: Another project can install SpeechDamageBench and regenerate the same damaged clip from a manifest entry.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Objective audio measurements

LEARN

- RMS and peak level.
- Simple SNR calculation when the clean reference is known.
- Spectral distance intuition.
- Why perceptual speech quality is not fully captured by one scalar metric.

BUILD IN MENDSPEECH

- Add baseline metrics for clean versus corrupted pairs.
- Store results in a tidy CSV schema with clip id, corruption, severity, seed, and measurements.

EXPERIMENT AND MEASURE

- Run all corruption levels on at least ten clips.
- Look for cases where a metric disagrees with your listening judgment.

REQUIRED OUTPUT

- `src/metrics/audio_metrics.py`
- `results/week1_damage_metrics.csv`
- `docs/metric_limitations.md`

Completion check: You can explain what each metric says and what it fails to say.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Build the first MendSpeech audio console

LEARN

- Audio playback in a lightweight interface.
- Waveform and spectrogram synchronization.
- Before and after comparison design.

BUILD IN MENDSPEECH

- Build a local page or notebook dashboard with clean and damaged playback.
- Add corruption controls and immediately regenerate the damaged clip.
- Display waveform, spectrogram, and basic measurements.

EXPERIMENT AND MEASURE

- Test with three speakers and several corruption types.
- Write down usability problems that would block later live debugging.

REQUIRED OUTPUT

- app/audio_lab.py
- screenshots/week1_audio_console.png

Completion check: Another person can open the tool, damage an utterance, and understand the visual change without reading your code.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Review, explain, and freeze Week 1

LEARN

- Review waveform, STFT, Mel features, SNR, clipping, dropouts, and reverberation.

BUILD IN MENDSPEECH

- Clean repository structure.
- Freeze SpeechDamageBench v0.1 severity presets and at least ten clean reference clips.
- Tag the benchmark package schema and add a minimal usage example independent of MendSpeech.

EXPERIMENT AND MEASURE

- From a blank notebook, recreate one corruption and one log Mel plot without copying previous cells.

REQUIRED OUTPUT

- reports/week1_audio_foundations.md
- data/week1_reference_manifest.csv
- speechdamagebench/README.md
- speechdamagebench/VERSION

Completion check: You can teach the complete path from clean waveform to controlled corruption and feature tensor.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks